

RBDC Ex 4

Project Brief: The Simplest Job Board for Job Seekers and Employers

Overview

This project focuses on building a minimal **job board** where employers can **post jobs** and job seekers can **search and apply**. It emphasizes clean and modular code, **persistent data management using PostgreSQL**, and **Dockerized deployment**. **Backend testing** will follow **TDD practices**, requiring **70% coverage with 100% tests passing**, while **frontend tests** are not required at this stage.

The goal is to create a maintainable and scalable solution aligned with **modular design standards** and industry best practices, focusing on efficient workflows for job posting, searching, and application submission.

Key Success Factors for Job Boards

1. Ease of Use

Employers: Ability to post jobs and manage applications seamlessly.

Job Seekers: Simple navigation, search, and application submission.

Operational Efficiency

Maintainable code following **Single Responsibility Principle** and **DRY** practices.

Dockerized deployment for easy setup and consistent environments.

3. TDD-driven Code Quality

High test coverage ensures reliability and minimizes bugs during future iterations.

Core Mechanisms Behind Job Boards

1. Job Posting Workflow

Employers create job posts containing **title, description, location,** requirements and status. The backend stores these posts in **PostgreSQL** and ensures they are retrievable by the frontend.

2. Application Submission Workflow

Job seekers fill out an **application form** with their name, email, and a cover letter. Applications are stored in the database, associated with the relevant job post, and accessible to employers.

3. Search & Filtering

Provide job seekers with **search and filter options** to find relevant jobs by title, category and location. Sort by Date posted, Relevance and Salary. Backend APIs will handle the logic for these queries.

TDD and Modular Code Practices

1. TDD Workflow:

- a. Write a failing test.
- b. Implement code to pass the test.
- c. Refactor while keeping the tests green.

2. Backend Testing:

Achieve **at least 70% coverage** with **100% tests passing**.

Write tests for **models, views, and API endpoints**.

3. Modular Code Design:

Use **SRP (Single Responsibility Principle)**: Each function and component should handle one thing well.

Follow **DRY (Don't Repeat Yourself)** practices.

Maintain a **separation of concerns**: Keep business logic, UI, and data management independent.

Project Structure and Tech Stack

Frontend (Next.js 14)

RBDC Ex 4 2

1. Components to Implement:

JobList: Displays job listings.

JobForm: Allows employers to post new jobs.

ApplicationForm: Allows job seekers to apply for jobs.

2. Testing:

Frontend tests are **not required** for now. Focus on clean, modular components.

Backend (Django 5.1)

1. Core Models:

Job: Stores job title, description, location, and posting time.

Application: Stores applicant name, email, and cover letter linked to a job post.

Employer: Stores employer name and contact information.

2. Sample Django Model Test:

```
python
Copy code
from django.test import TestCase
from .models import Job

class JobModelTest(TestCase):
    def setUp(self):
        self.job = Job.objects.create(
            title="Frontend Developer",
            description="Build responsive UIs",
            location="Remote"
        )
```

```

def test_job_creation(self):
    self.assertEqual(self.job.title, "Frontend Developer")

    self.assertEqual(self.job.location, "Remote")

```

RBDC Ex 4 3

Database (PostgreSQL)

Schema Design:

sql

Copy code

```

CREATE TABLE jobs (
    id SERIAL PRIMARY KEY,
    title VARCHAR(255),
    description TEXT,
    location VARCHAR(255),
    posted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );

```

```

CREATE TABLE applications (
    id SERIAL PRIMARY KEY,
    job_id INTEGER REFERENCES jobs(id),
    applicant_name VARCHAR(255),
    applicant_email VARCHAR(255),
    cover_letter TEXT,
    applied_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );

```

Containerization with Docker

1. Dockerfile for Django Backend:

dockerfile
Copy code
FROM python:3.9-slim

RBDC Ex 4 4

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .

CMD ["gunicorn", "--bind", "0.0.0.0:8000", "myproject.wsg i:application"]

2. Docker Compose Setup:

yaml
Copy code
version: '3.8'

services:
 backend:
 build: ./backend
 ports:
 - "8000:8000"
 environment:
 - POSTGRES_USER=postgres
 - POSTGRES_PASSWORD=password

postgres:
 image: postgres:13
 ports:
 - "5432:5432"
 environment:
 POSTGRES_USER=user
 POSTGRES_PASSWORD=password

Testing with Postman

Sample API Request:

json

Copy code

POST /api/jobs

Content-Type: application/json

```
{  
  "title": "Software Engineer",  
  "description": "Develop amazing features",  
  "location": "Remote"  
}
```

Deliverables and Submission

1. GitHub Repository:

Backend code with **at least 70% test coverage** and **100% tests passing**.

Modularized code following SRP and DRY principles.

2. Dockerized Deployment:

Use Docker to containerize the frontend, backend, and PostgreSQL services.

3. Documentation:

Provide **setup instructions**, **API documentation**, and **testing guidelines**.

4. Live Deployment:

Deploy the project on **AWS EC2**.

This project reinforces **TDD practices**, **modular coding**, and **efficient workflows** for job posting and application submission. Good luck, and happy coding! 💎💎

RBDC Ex 4 6